

TESI FINALE DI PROGETTAZIONE DI SISTEMI OPERATIVI

Programmazione concorrente nel Linguaggio Java

Approfondimento sull'utilizzo dei *Thread* in Java

Studente:

Tommaso Cosimi

Matricola 191739

Docente:

Prof.ssa Letizia Leonardi

Codocente:

PhD Silvia Cascianelli

Sommario

Nel corso della Storia dell'Informatica, l'avanzamento tecnico e tecnologico verificatosi nell'ambito delle Architetture dei Calcolatori e dei Sistemi Operativi ha permesso grandi passi in avanti nelle dinamiche di utilizzo degli Elaboratori Elettronici. Si è assistito al passaggio dall'esecuzione sequenziale delle operazioni ad un'esecuzione parallela delle stesse, il tutto all'interno di un singolo Elaboratore.

L'opportunità di avere esperienze utente multiprogrammate è stata resa possibile, oltre che dagli Elaboratori non sequenziali, anche da Linguaggi di Programmazione anch'essi non sequenziali che permettano l'esecuzione concorrente di più processi. Tale flessibilità ha permesso un grado di utilizzo delle risorse nettamente superiore al passato, permettendo prestazioni superiori a parità di risorse.

Lo scopo di questo elaborato sarà quello di approfondire l'argomento della Programmazione Concorrente nel Linguaggio Java, riportando allo stesso tempo esempi pratici di utilizzo.

Indice

1	Introduzione	11
1.1	Algoritmo, Programma, Processo	11
1.2	Esecuzione Sequenziale e non Sequenziale	13
1.2.1	Processi Sequenziali	14
1.2.2	Processi non Sequenziali	14
1.3	Processi Pesanti e Processi Leggeri	15

Elenco delle figure

1.1	Grafo delle Precedenze di un Processo Sequenziale.	14
1.2	Grafo delle Precedenze di un Processo non Sequenziale. . . .	15

Elenco delle tabelle

Elenco degli algoritmi

1	<i>Bubble Sort</i>	12
---	------------------------------	----

Capitolo 1

Introduzione

Il seguente Capitolo fornirà un'introduzione teorica al concetto di esecuzione non sequenziale di un processo, ponendo enfasi sui vantaggi portati e sulle nuove necessità richieste da tale implementazione.

1.1 Algoritmo, Programma, Processo

Nella definizione della singola unità di esecuzione si ritiene opportuno effettuare una leggera digressione.

Algoritmo

Un Algoritmo è un procedimento puramente logico studiato appositamente per la risoluzione di uno specifico problema. In letteratura sono rintracciabili moltissimi Algoritmi, per brevità vengono nominati a titolo di esempio Algoritmi di Ordinamento come il *Bubble Sort*[1] - di cui viene fornito lo pseudocodice a seguire - od il *Radix Sort*[2], e Algoritmi di Ricerca del Cammino Minimo in un Grafo come l'*Algoritmo di Dijkstra*[3] o l'*Algoritmo di Bellman-Ford*[4].

Algoritmo 1: Ordinamento per confronti “*Bubble Sort*”

```

Dati: A;                // Vettore di numeri interi non ordinati
1 partizioneNonOrdinata  $\leftarrow$  A.length;
2 finché partizioneNonOrdinata > 1 fai
3   per i = 2 fino a k fai
4     se A[i] < A[i - 1] allora
5       |   Scambia A[i] ed A[i - 1];
6     fine se
7   fine per
8   partizioneNonOrdinata  $\leftarrow$  (ultimoScambio - 1);
9 fine finché
10 ritorna A;            // Vettore di numeri interi ordinati

```

Programma

Un Programma è la descrizione in codice di un Algoritmo in uno specifico Linguaggio di Programmazione in maniera tale da permetterne l'esecuzione all'interno di un Calcolatore, è un'entità statica. Si riporta di seguito un'esempio di Programma che descriva l'Algoritmo di Ordinamento per confronti “*Bubble Sort*” implementato tramite il Linguaggio di Programmazione Java.

```

1 public class BubbleSort {
2
3     public static void BubbleSort(int[] A) {
4
5         // Definizione della dimensione della partizione del
6         //  $\hookrightarrow$  vettore non ordinata
7         int unorderedPartition = A.length;
8
9         // Ordinamento
10        while(unorderedPartition > 1) {
11            // Scorro la parte non ordinata
12            for(int i=2; i<unorderedPartition; i++) {
13                // Se non e' ordinato, scambia
14                if(A[i] < A[i-1]) {
15                    int buf = A[i];
16                    A[i] = A[i-1];
17                    A[i-1] = buf;
18                }
19            }
20            unorderedPartition--;
21        }
22    }
23 }

```

```
16         A[i-1] = buf;
17     }
18 }
19     // Dopo lo scambio, riduco la dimensione della
    ↪ partizione del vettore non ordinata
20     unorderedPartition = unorderedPartition - 1;
21 }
22
23 }
24
25 }
```

Processo

Un Processo è la sequenza di operazioni eseguite da un Elaboratore che opera sotto il controllo di un Programma, è un'entità dinamica: in maniera intuitiva è possibile affermare che in linea generale un Processo è un Programma in esecuzione. Di seguito si riporta un esempio di esecuzione sequenziale di due Processi che eseguono lo stesso Programma, ovvero *Bubble Sort*.

```
1 [tcosimi@VivoBook-Arch]:$ java BubbleSort 1 2 3 6 5 2
2 Il vettore non ordinato e': [1, 2, 3, 6, 5, 2]
3 Il vettore ordinato e': [1, 2, 2, 3, 5, 6]
4 [tcosimi@VivoBook-Arch]:$ java BubbleSort 1 9 8 3 6 7
5 Il vettore non ordinato e': [1, 9, 8, 3, 6, 7]
6 Il vettore ordinato e': [1, 3, 6, 7, 8, 9]
```

1.2 Esecuzione Sequenziale ed Esecuzione non Sequenziale

La sequenza di operazioni eseguite da un processo può essere rappresentata attraverso un Grafo Orientato che indichi la precedenza delle une sulle altre. In questa visualizzazione i nodi indentificano il singolo evento da verificarsi, mentre gli archi l'ordinamento temporale tra i nodi.

1.2.1 Processi Sequenziali

Facendo riferimento alla rappresentazione descritta in precedenza, un Processo Sequenziale è tale da poter essere schematizzato tramite l'ausilio di un Grafo ad Ordinamento Totale: una particolare tipologia di Grafo ove ogni nodo - e quindi ogni evento - fatta eccezione per il nodo di inizio ed il nodo di fine, possiede un solo predecessore ed un solo successore.

A seguire viene effettuato un esempio di Processo Sequenziale.

Esempio di Rappresentazione di un Processo Sequenziale

Si riporta a titolo di esempio un Processo che richiede all'utente due numeri interi e ne restituisce la somma. Esso è rappresentabile come una sequenza ordinata di operazioni atomiche come esplicitato a seguire.

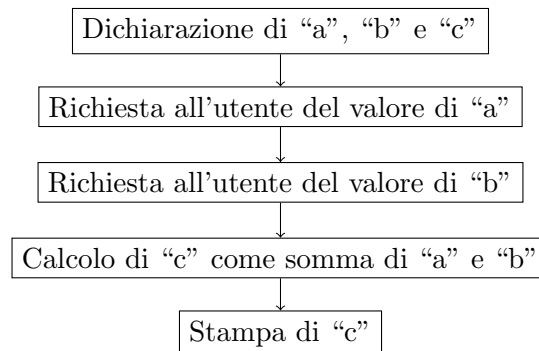


Figura 1.1: Grafo delle Precedenze di un Processo Sequenziale.

1.2.2 Processi non Sequenziali

Al contrario della controparte, un Processo non Sequenziale può essere rappresentato tramite l'ausilio di un Grafo ad Ordinamento Parziale: ogni nodo interno potrà avere più predecessori e più successori. Questo è possibile in quanto alcune operazioni eseguite dal Processo sono indipendenti tra loro o non è interessante il loro ordine di terminazione per l'utente finale.

A seguire viene effettuato un esempio di Processo non Sequenziale.

Esempio di Rappresentazione di un Processo non Sequenziale

Si supponga di dover valutare l'espressione matematica:

$$(5 + 8) + (6 - 3) * (9 + 2)$$

Ovviamente è possibile eseguire sequenzialmente la valutazione verificando prima i risultati delle operazioni tra parentesi, poi la moltiplicazione tra gli ultimi due termini ed infine l'addizione tra i primi due.

La natura del problema permette tuttavia di eseguire in ordine arbitrario dei sottoinsiemi di operazioni, i quali risultati risultano per giunta indipendenti da un punto di vista computazionale: è possibile in effetti calcolare in ordine indifferente o addirittura delegare la valutazione a differenti agenti di calcolo le espressioni elementari all'interno delle parentesi.

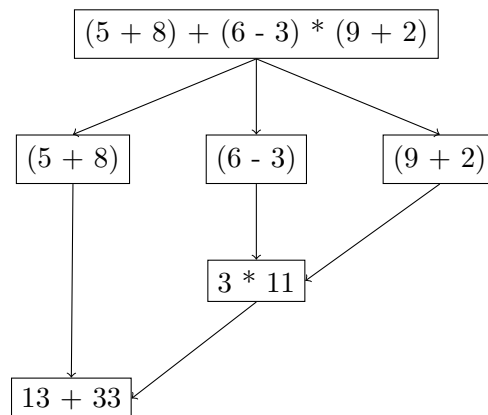


Figura 1.2: Grafo delle Precedenze di un Processo non Sequenziale.

1.3 Processi Pesanti e Processi Leggeri

All'interno di un Sistema Operativo è possibile individuare due tipologie di Processi in esecuzione, ognuna di esse con caratteristiche distintive:

- Processi Pesanti;
- Processi Leggeri.

Processi Pesanti

I Processi Pesanti sono identificati tramite la ennupla:

$$\{Program\ Counter, Registri, Stack, Codice, Dati\}$$

Processi Pesanti distinti eseguono codice distinto ed accedono a dati distinti, non condividendo memoria ed ispirandosi quindi al Modello ad Ambiente Locale.

Processi Leggeri

I Processi Leggeri - o *Thread* - sono invece identificati dalla ennupla:

$$\{Program\ Counter, Registri, Stack\}$$

Più *Thread* correlati da un punto di vista logico condividono un'area di dati e codice tra loro, formando un *Task*. Ciascun Processo Leggero rappresenta differenti contesti di esecuzione che risultano indipendenti all'interno di uno stesso *Task*. Da tutto ciò segue che i *Task* sono invece identificati da una ennupla del tipo:

$$\{Thread_1, Thread_2, \dots, Thread_n, Codice, Dati\}$$

Alla luce di quanto illustrato è possibile immaginare un Processo Pesante come un *Task* composto da un solo *Thread*.

Si noti come sia comunque necessario, ai fini del *Context Switching*, mantenere un Descrittore di Processo (*Process Control Block* - *PCB*) per ogni Processo Leggero, rendendo più complesso il *PCB* del *Task* che va ad incorporare anche quelli dei suoi relativi *Thread*.

In termini implementativi, i *Thread* di uno stesso *Task* condividono lo spazio di indirizzamento, le variabili globali e la tabella dei File aperti proprie del *Task*, ispirandosi quindi al Modello ad Ambiente Globale. Questa somiglianza con tale ambiente implica l'insorgenza di problematiche quali le cosiddette "*Race Conditions*" date da interazioni di tipo sia competitivo che cooperativo: vanno di conseguenza implementate misure atte alla sincronizzazione tra i vari Processi Leggeri del *Task*.

Bibliografia

- [1] Wikipedia. Bubble Sort, Giugno 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Bubble_sort. Pagina visitata il 5 novembre 2023.
- [2] Wikipedia. Radix Sort, Maggio 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Radix_sort. Pagina visitata il 5 novembre 2023.
- [3] Wikipedia. Algoritmo di Dijkstra, Aprile 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Algoritmo_di_Dijkstra. Pagina visitata il 5 novembre 2023.
- [4] Wikipedia. Algoritmo di Bellman-Ford, Maggio 2023. Articolo disponibile all'url: https://it.wikipedia.org/wiki/Algoritmo_di_Bellman-Ford. Pagina visitata il 5 novembre 2023.